



DEPI final project documentation

This project is done by:

Mohamed Ahmed Doma

Ahmed Tamer Ahmed

Ahmed Ibrahim Abo Shanab

Moaz El Gendy

Ahmed Mohamed Makram

Ahmed Hossam

Under the supervision of :

ENG. AHMED GAMIL OSMAN



Agenda:

What is the project?

What are the steps used to achieve this project?

What are the tools used in the project?

How did you use these tools in your project?



1) What is the project?

It is about Applying the **DevOps** concepts that we learned about in DEPI third round from the ministry of communication and information technology, we applied the concepts of devops on a webapplication made with react framework, nodejs.

-What is the application about?

The application is healthcare web application that you can sign up for to make an appointment without the need for you to go to the clinic and waste your time, the web application is user friendly and easy to understand and use.

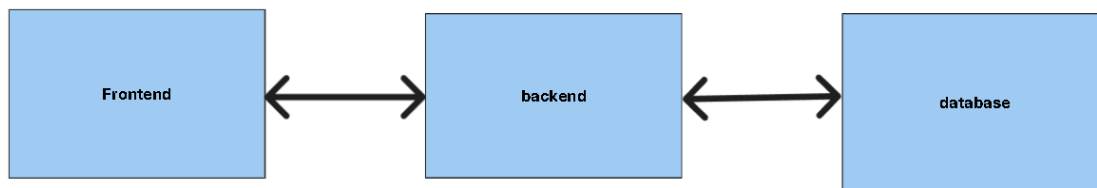
-What architecture are we using? and why?

We are used the **micro services** architecture in our web application because it is the best practice for applying **DevOps** concepts.

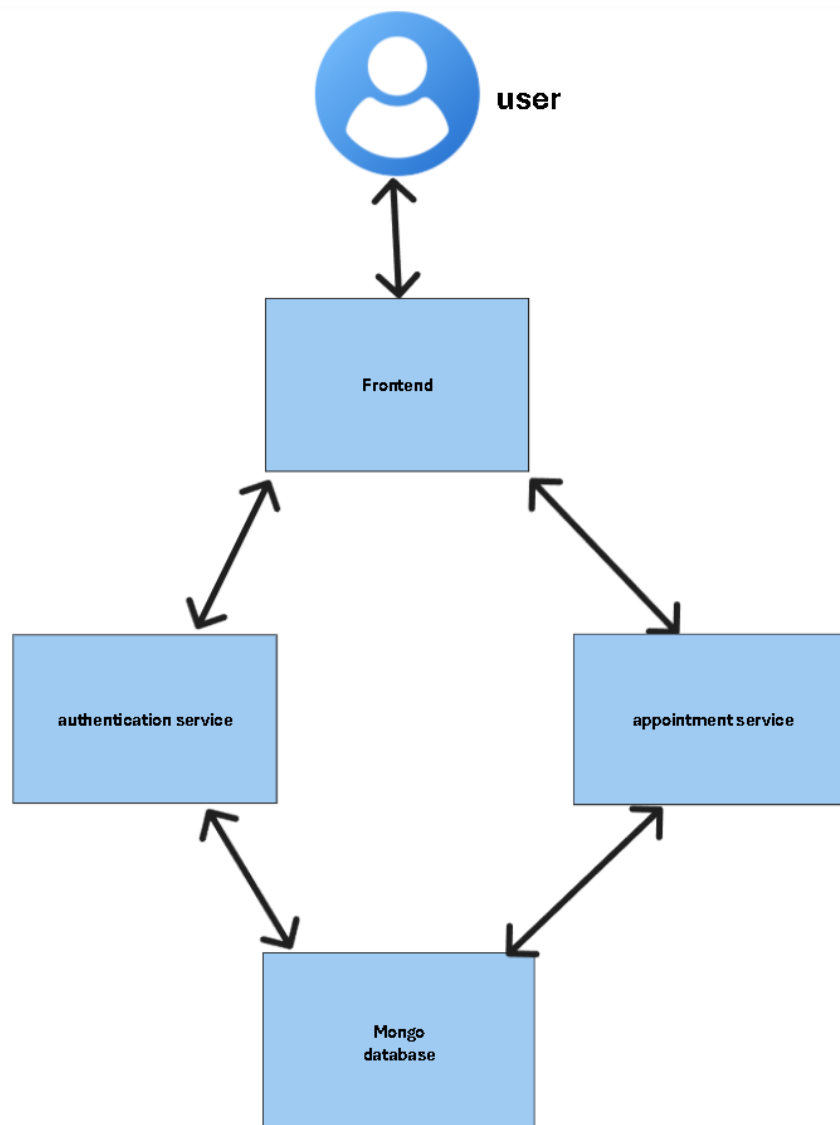
-What are the the microservices that you use?

We are using frontend service, authentication service (backend), appointment service (backend), and database service (mongodb).

-What is the general way of communication do you use in your application?



-What is the general way of communication between your services?





2) What are the steps used to achieve this project?

First: setting up the environment and tools

- WSL UBUNTU VM
- DOCKER DESKTOP
- VS CODE
- CONNECTING GITHUB TO VSCODE

Second: programming the web application from scratch

We made the web application from scratch using the reactjs and nodejs.

Third: testing the application using

We made sure that the whole web application is working.



3) What is the tools used in the project?

In this project we used tools for coding like:

-VS CODE

VMs like:

UBUNTU LINUX

Containerization tools like:

-DOCKER DESKTOP

Orchestration tools like:

-KUBERNETES

CI/CD tools like:

-GITHUB ACTIONS



4) How did you use these tools in your project?

1) Isolation using Dockers:

We made images of each microservice of our web application by making dockerfiles, this helped us in isolating each service in the web application.

2) networking test using Docker compose:

Since the microservices are isolated from each other, we made a docker compose file to allow communication between the containers through a network between those containers only. We didn't use mongo data base on our vscode instead we used the docker **official image** of mongo db from Dockerhub.

3) Orchestrating services using Kubernetes (K8S):

Since we have web application with a critical job we needed a way to make our application flexible (scalable), so we used K8s orchestrate our application containers and network them together.

-steps of orchestration:

- we made two yaml files for each service one for **service** (internal communication between pods that contains services containers), and other yaml file for **deployment** to limit the usage for hardware for each pod.
- To show the application outside the K8s's cluster **ingress** was used, only the front end is shown but the other services are not exposed outside the cluster.
- Since ingress is only a map of where each service is an **ingress controller** is needed to forward the requests from outside the cluster to each pod inside the cluster. We used nginx ingress controller.



4) CI/CD pipeline:

If anything was to be edited in the code of the web application there is a must to update our docker images then push then again to the docker hub so that k8s can then pull those new images if the cluster was sleep and then woke up but if the cluster was already running then refreshing the pods or applying all the K8s yaml files again is a must so that the pods get the new docker images and show the new feature that was added to the web application. These are a lot of commands to be run (building docker images, pushing images to docker hub, refreshing the cluster and the pods) so that is why we made a CI/CD pipeline using **github actions**. The github action pipeline in our project is divided into two parts.

- **CI (Continuous Integration):**
This part is done on the github cloud where it builds and pushes the docker images to the docker hub.
- **CD (Continuous Deployment):**
This part is done on the pc where the wsl is to refresh the K8s cluster each time a new docker image is pushed to the docker hub, it is done by downloading a github actions local runner which allow refreshing the cluster to get the new docker images.

The CI/CD pipeline starts working when the code is pushed to github.

5) Monitoring using promethues and grafana:

When running a cluster we don't see anything except the frontend of the application we exposed from the cluster and to see what is inside the cluster we will use a bunch of commands. So the most simple to see (not doing) is to use monitoring tools like promethues and it is a tool that scrapes data from the pods in the cluster so to do this there are steps to based on the type of the service the pods is showing. In our project since that only exposed service is frontend we will not be able to scrape data from the other pods so the solution is to make a monitoring pods in the cluster to scrape the data from the pods then show the data on promethues (and that is what we did). Looking to data which are only numbers is exhausting but there is a way to change its view which is by using the tool **Grafana**, this tool get the data scraped from promethues and shows them in graphs and other design according to what you love.

Final project diagram:

